Workshop B

MF2007

David Demir, Hsiu-Fang Chien, Johan Kindlundh
Group 11

February 3, 2026

Table of Contents

1 Servo Control **1**
1.1 Trajectory planner 1
1.2 Model following controller 2
1.3 Implementation 2
1.4 Comparing Simulation with Real Motor 3

2 Writing Controller Code **9**
2.1 Implementation 9
2.2 Comparing Embedded Controller with Block Diagram Approach 15

3 Robustness to Parameter Uncertainty and Sensor Noise **19**
3.1 DC motor 19
3.2 Hydraulic Actuator 22

1 Servo Control

1.1 Trajectory planner

To design the trajectory planner, the maximum torque first needed to be computed using the following formula,

$$\tau_{\max} = K_m \cdot \frac{U_{\max}}{R}, \tag{1.1}$$

where U_{max} is the maximum voltage that the driver can supply to the motor (24 V), K_m is the motor constant and R is the motor's internal resistance. Second, the acceleration of the motor was calculated using

$$-a_- = a_+ \leq \frac{\tau_{\max} - dv_1 - T_c}{J_{\text{tot}}}, \tag{1.2}$$

where, it was assumed that the deceleration had the same magnitude as the acceleration, but with inverted signs. Here, T_c is the Coulomb torque and J_{tot} is the total inertia of the system, consisting of the internal motor inertia and load inertia. v_1 is a tunable parameter chosen to be 250 rad/s since it was the achievable angular velocity that the motor could reach when supplied with 20 V on Workshop A.

Using these values, the trajectory path function for the acceleration could be described, as is seen below.

$$a(t) = \begin{cases} 0, & t = 0, \\ a_+, & 0 < t \leq t_1, \\ 0, & t_1 < t < t_2, \\ a_-, & t_2 \leq t < t_f, \\ 0, & t_f \leq t \end{cases} \tag{1.3}$$

By means of integration, the velocity and the final reference position of the trajectory planner could be derived, which are shown below.

$$v(t) = \begin{cases} a_+ t, & 0 \leq t < t_1, \\ a_+ t_1, & t_1 \leq t < t_2, \\ a_+ t_1 + a_-(t - t_2), & t_2 \leq t \leq t_f \end{cases} \tag{1.4}$$

$$r(t) = \begin{cases} \frac{1}{2} a_+ t^2, & 0 \leq t < t_1, \\ a_+ t_1 t - \frac{1}{2} a_+ t_1^2, & t_1 \leq t < t_2, \\ a_+ t_1 t - \frac{1}{2} a_+ t_1^2 + \frac{1}{2} a_-(t - t_2)^2, & t_2 \leq t \leq t_f \end{cases} \tag{1.5}$$

1.2 Model following controller

By expanding the nonlinear motor differential equations and isolating the controller input, a model following controller could be created as per Equation 1.6,

$$u_{ff}(t) = \frac{R}{K_m}(J_{tot}a(t) + T_c sign(v(t)) + d_m v(t)), \tag{1.6}$$

where $u_{ff}(t)$ is the model following control input for the plant, and the input values $a(t)$ and $v(t)$ were provided from the trajectory planner to create a reference model. The reference model defined the ideal dynamics the system was intended to achieve, and the controller adjusted the inputs to the plant to minimize the difference between the model's output and the plant's output.

The model following controller was paired with an output feedback PID-controller, supplying another controller input u to the plant, to compare the actual system state to the desired state and make adjustments accordingly. Without a controller, there was no mechanism to reduce error or account for disturbances.

1.3 Implementation

The trajectory planner, model following controller and the PID-controller were implemented into a block diagram as can be seen in Figure 1.

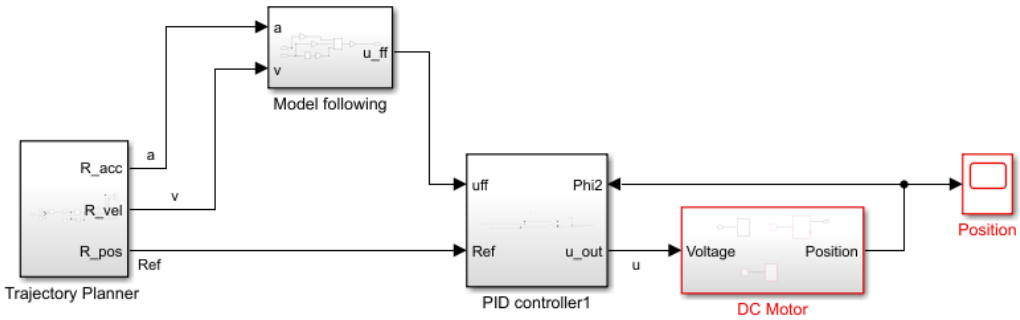


Figure 1: Block diagram of the whole control implementation.

This resulted in the following control law in the continuous s-domain,

$$u(s) = \frac{T}{R}ref(s) + \frac{S}{R}pos(s) + \mathscr{L}\{u_{ff}(t)\}, \tag{1.7}$$

where T, R, and S are the same notations used in the Diophantine equation for the feedback and feedforward parts of a 2-Degrees of Freedom (DoF) output feedback controller. $u(s)$ is the input to the plant, $ref(s)$ is the reference/setpoint value and $pos(s)$ is the model output (angular position). $\mathscr{L}\{u_{ff}(t)\}$ is the Laplace transform of the model following control output $u_{ff}(t)$ to the s-domain.

1.4 Comparing Simulation with Real Motor

In Figure 2, the simulation somewhat aligns with the real motor when subjected to a 10 *rad* set point value. The difference is likely due to the modeling errors of the simulation.

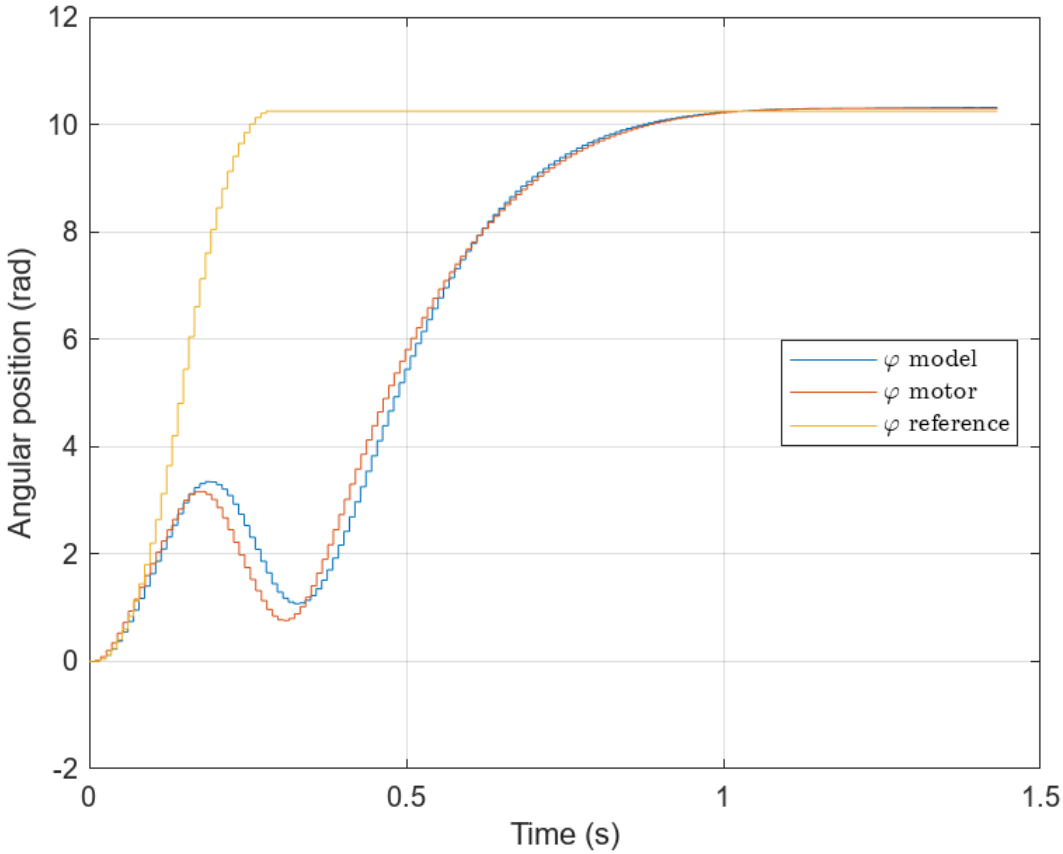


Figure 2: Trajectory for 10 *rad* set point, showing both model, motor, and reference trajectory path.

However, it is evident that the trajectory position is not exactly, 10*rad*. This is likely due to the discrete integration in the trajectory planner, accumulating a small error in the final position value, and the sampling time not aligning with the planned trajectory, causing the acceleration to be continued for as much as one sample longer than the planned trajectory. The trajectory was determined using a look-up table in Simulink, as shown in Figure 3.

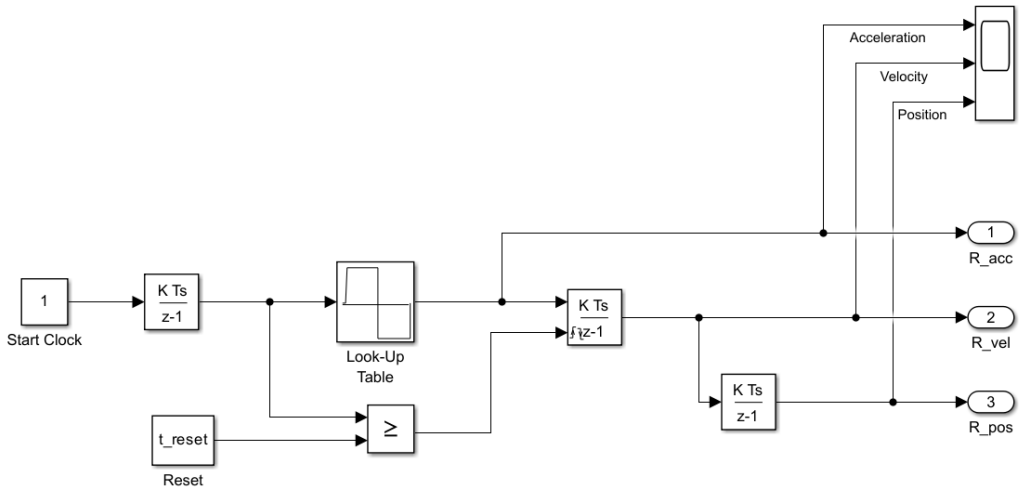


Figure 3: Block diagram for the trajectory planner.

The trajectories for the 10 *rad* set point are demonstrated in Figure 4.

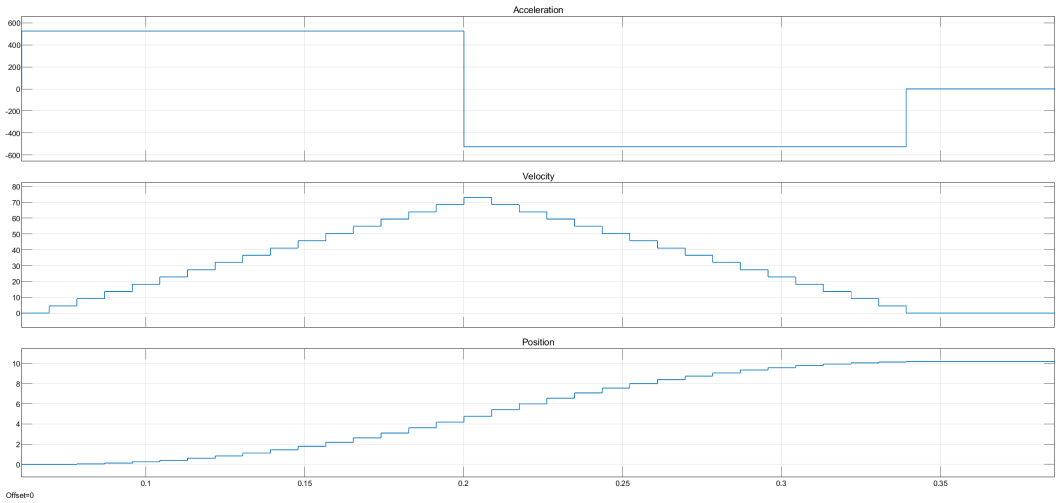


Figure 4: Trajectories for acceleration, velocity and position for a 10 *rad* set point.

The controller output is shown in Figure 5.

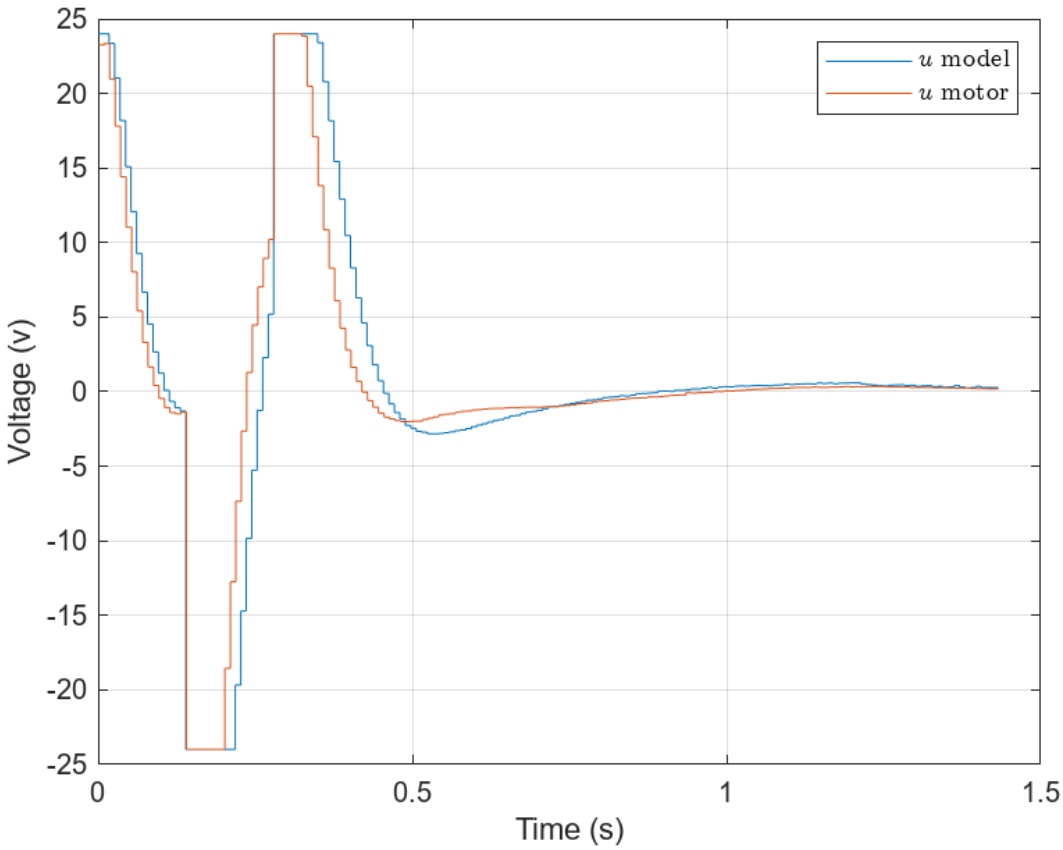


Figure 5: Controller output for 10 *rad* set point, showcasing both simulated output and output with real motor in the loop

For both simulation and the real motor, the trajectory path is not followed as expected. Because this profile even shows in simulation, it suggests the behavior is not due to modeling error, but is instead related to the assumptions made when designing the trajectory planner. One example of such an assumption is that the motor would be able to have infinite jerk, as the trajectory is a_+ at $t = 0$. Furthermore, a compounding issue is that the model follower controller does not cover the physics of the plant completely, and therefore does not supply sufficient voltage to keep it on the trajectory reference path. Another factor is that the trajectory planner amuses symmetrical acceleration and deceleration, while the model following controller does not, further compounding this issue. This leads to the trajectory planner reaching the set point value before the plant and making the PID take over, resulting in the profile in Figure 5.

However, as in Figure 6, with a larger set point, the model following control experiences less of this behavior.

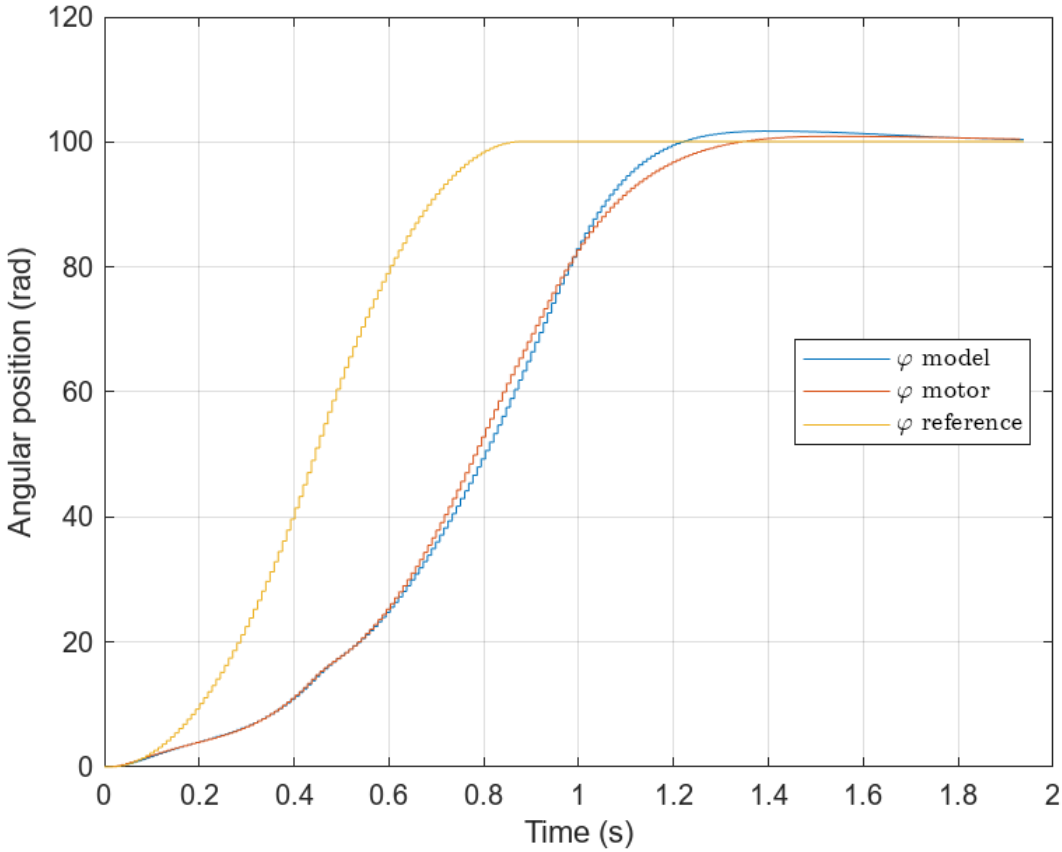


Figure 6: Trajectory for 100 *rad* set point, showing both model, motor, and reference trajectory path.

This discrepancy is likely due to the longer acceleration phase and the fact that a larger error has been accumulated, making the PID produce a larger control output in relation to the model following controller. The trajectories for 100 *rad* set point are given by Figure 7.

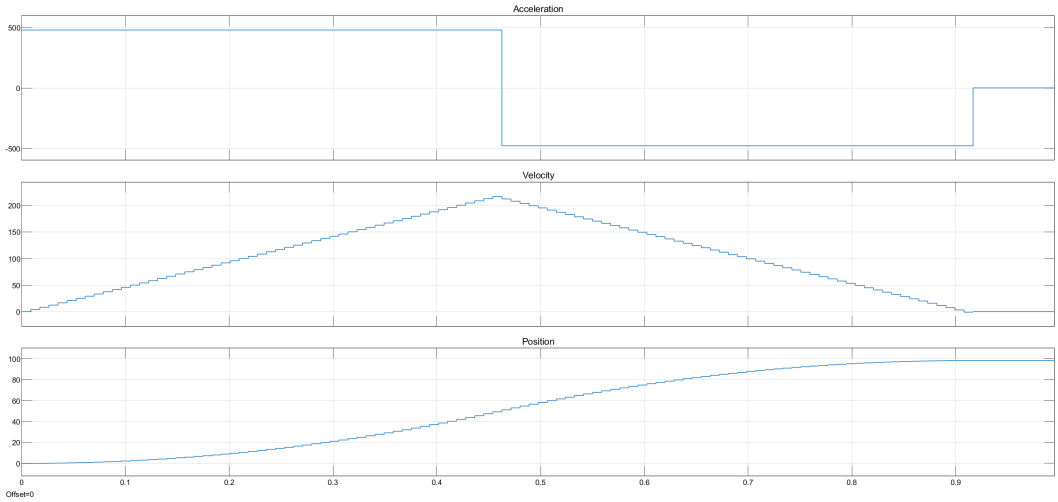


Figure 7: Trajectories for acceleration, velocity and position for a 100 *rad* set point.

By analyzing the control output in Figure 8, it is apparent that it saturates more frequently at higher set points.

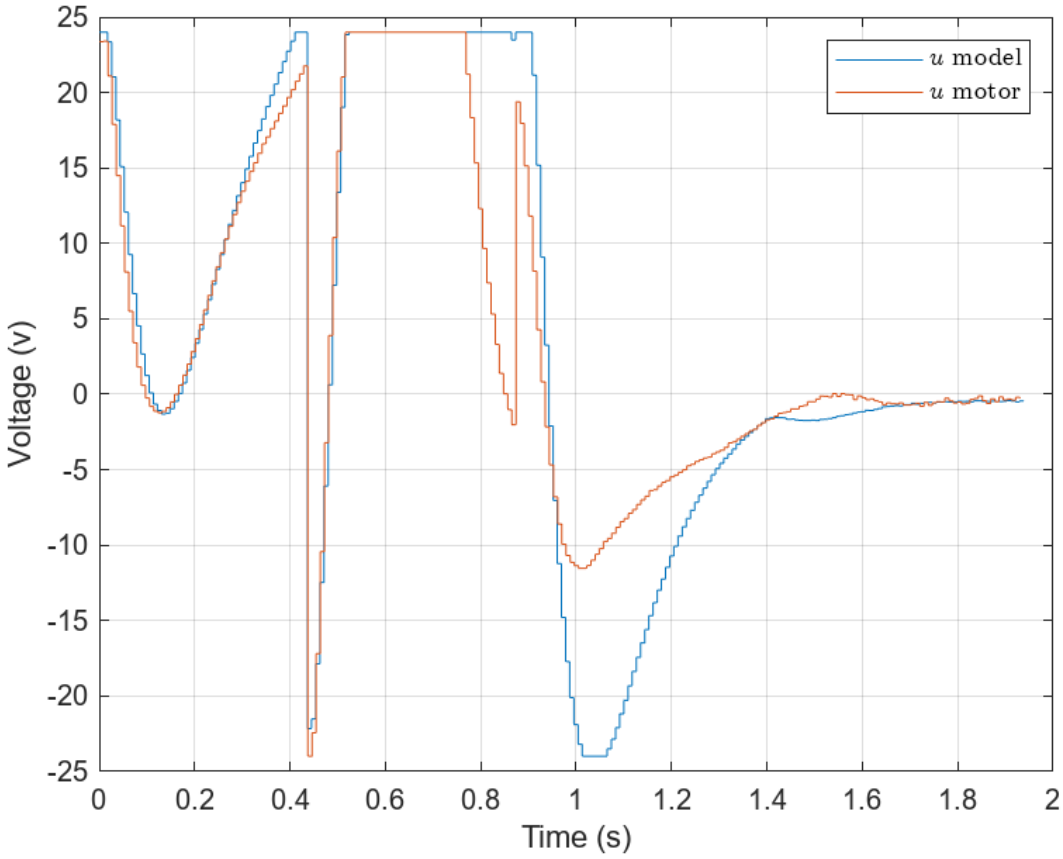


Figure 8: Controller output for 100 *rad* set point, showcasing both simulated output and output with real motor in the loop

However, this largely depends on how aggressive the trajectory planner is designed to be. The tuning parameter v_1 is mainly what determines this aggression, i.e. the constant velocity that the trajectory planner settles in between the acceleration and deceleration phases. However, a constant velocity is never reached during this experiment as shown in Figure 7.

2 Writing Controller Code

2.1 Implementation

Figure 9 illustrates the embedded controller and its block diagram structure, similar to the one in Section 1.3.

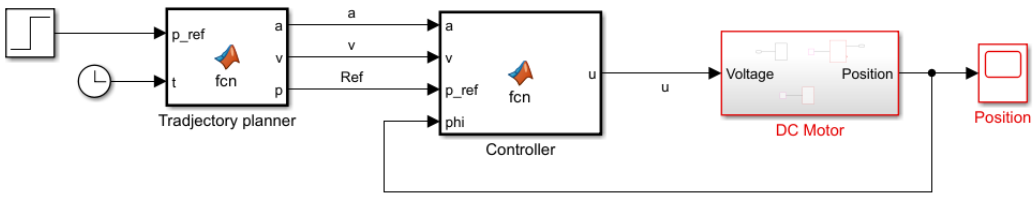


Figure 9: Block diagram of the embedded controller.

To develop the embedded controller, it initially computes the acceleration, velocity, and position trajectories for the next time step. Depending on the set point values, the controller uses different trajectories: one requires a constant speed between acceleration and deceleration phases, and the other does not. Regardless, the information is then transferred to the second function block that integrates both model-following control and the PID controller from Workshop A. The trajectory planner code is presented below.

```
function [a, v, p] = fcn(p_ref, t)
    %t is the real time
    %p_ref is the setpoint
    persistent t1 t3 t2 vmax q a_plus r_accel tf;
    if isempty(t1)
        K_m = 69.7e-3;                  % Motor constant
        R = 112;                        % Resistance
        J_m = 7.46e-7;                  % Motor inertia
        J_l_val = 2.7803e-5;            % Load inertia
        J_tot_val = J_m + J_l_val;      % Total inertia
        T_c_val = 4.1e-4;               % Coulomb torque
        max_voltage = 24;               % Max voltage
        d_m_nonlin = 3.55e-6;           % Dynamic friction
        a=single(0); v=single(0); p=single(0); %initiation parameters
        max_torque = K_m * (max_voltage / R); %Max torque supplied from the motor
        v1=250;                         % Max velocity (tunabel)
```

```
        a_plus = single((max_torque -d_m_nonlin*v1- T_c_val) / J_tot_val); % Max acceleration
        vmax= single(v1);

        % Check if vmax can be reached
        t1 = single(vmax / a_plus);            % Time to reach vmax
        r_accel = single(0.5 * a_plus * t1^2); % Position during full acceleration

            if 2 * r_accel >= p_ref
            % Cannot reach vmax: Two-phase trajectory
            t1 = single(sqrt(p_ref / a_plus));
            t3 = t1;
            tf = t1+ t3;                       % Total time
            vmax=a_plus*t1;
            q=int8(1);
            t2=t1;
            else
                % Can reach vmax: Three-phase trajectory
                r_const = single(p_ref - 2 * r_accel);          % Remaining position at constant velocity
                t2 = single(r_const / vmax);                    % Time at constant velocity
                t3 = single(t1);                                % Deceleration time
                tf = t1 + t2 + t3;                              % Total time
            vmax=a_plus*t1;
            q=int8(0);
            end
    end
    if q==1
        % Generate trajectory
        if t <= t1
            % Phase 1: Acceleration
            a = a_plus;
            v= a_plus * t;
            p= 0.5 * a_plus * t^2;
```

```
        elseif t<tf
            % Phase 2: Deceleration
            decel_time = t - t1 ;
            a= -a_plus;
            v = vmax - a_plus * decel_time;
            p = 0.5 * a_plus * t1^2  + vmax * decel_time - 0.5 * a_plus * decel_time^2;
        else
            decel_time = tf - t1;
            a= single(0);
            v = single(0);
            p = 0.5 * a_plus * t1^2 + vmax * decel_time - 0.5 * a_plus * decel_time^2;
        end
    else

    % Generate trajectory
        if t <= t1
            % Phase 1: Acceleration
            a = a_plus;
            v= a_plus * t;
            p = 0.5 * a_plus * t^2;
        elseif t < t1 + t2
            % Phase 2: Constant velocity
            a= single(0);
            v= vmax;
            p= r_accel + vmax * (t - t1);
        elseif t<tf
            % Phase 3: Deceleration
            decel_time = t - t1 - t2;
            a= -a_plus;
            v= vmax - a_plus * decel_time;
            p= r_accel + vmax * t2 + vmax * decel_time - 0.5 * a_plus * decel_time^2;
        else
            decel_time = tf - t1 - t2;

            a= single(0);
            v = single(0);
            p = single(r_accel + vmax * t2 + vmax * decel_time - 0.5 * a_plus * decel_time^2);
        end
```

```
    end
    end
```

The function block takes two parameters, the reference position, r, and the simulation time. By calculating the acceleration, velocity and position, p, for the next time step in the simulation, this information is then utilized in the difference equation of the PID-controller in the second function block. To determine this difference equation, the control law of the PID-controller was defined in Workshop A as shown below.

$$u(z) = \frac{1.076(z - 0.0934)}{(z-1)(z+0.575)} r(z) - \frac{9.386z^2 - 13.63z + 5.126}{(z-1)(z+0.575)} p(z) \tag{2.1}$$

Since no additional tuning was applied to this controller, there remain room for improvement that could potentially yield better results. This will be evident when looking at the simulation results, particularly the controller output plot. Nevertheless, this equation were then be rewritten on the following form, where both the LFH and the RHS of the equation have been multiplied by z^{-2}.

$$\begin{aligned} u(z) = {} & 0.425z^{-1}u(z) + 0.575z^{-2}u(z) + 1.076z^{-1}r(z) \\ & + 0.100z^{-2}r(z) - 9.386p(z) + 13.63z^{-1}p(z) - 5.126z^{-2}p(z). \end{aligned} \tag{2.2}$$

From this, the difference equation for the PID-controller could be formulated, relating the current output of the system to its past outputs, current input and past inputs, using discrete time steps:

$$\begin{aligned} u_n = {} & 0.425u_{n-1} + 0.575u_{n-2} + 1.076r_{n-1} \\ & + 0.100r_{n-2} - 9.386p_n + 13.63p_{n-1} - 5.126p_{n-2}. \end{aligned} \tag{2.3}$$

Together with equation 1.6 which calculated the trajectory following control input $u_{ff}(t)$, the total control input for the plant is determined within the controller function block shown in Figure 9. The code for the function, detailed below, computes the control input for the next time step by using the current velocity, reference position, acceleration, and the feedback position from the plant output.

```
function u  = fcn(p_ref,v , a, phi)
    % Persistent variables to store past values
    persistent u_n1 u_n2 r_n1 r_n2 y_n1 y_n2;

    if isempty(u_n1)
        u_n1 = single(0); u_n2 = single(0);
        r_n1 = single(0); r_n2 = single(0);
        y_n1 = single(0); y_n2 = single(0);
```

```
    end
    K_m_val = 69.7e-3;
    R_val=112;
    J_m = 7.46e-7;                  % Motor inertia
    J_l_val = 2.7803e-5;            % Load inertia
    J_tot_val = J_m + J_l_val;      % Total inertia
    T_c_val = 4.1e-4;               % Coulomb torque
    max_voltage = single(24);       % Max voltage
    d_m_nonlin = 3.55e-6;
    uff=(a*J_tot_val+sign(v)*T_c_val+d_m_nonlin*v)*(R_val/
       K_m_val);

    % Controller coefficients
    R_num = [1, -1.577240775571420, 0.577240775571420];
    S_num = [39.241359035203010, -75.903796975848650,
       36.719178203195696];
    T_num = [1.587982040617708, -2.575622452129391,
       1.044380674061737];

    % Compute T(z)/R(z) * p_ref
    T_R = (T_num(1) * p_ref + T_num(2) * r_n1 + T_num(3) *
       r_n2) - (R_num(2) * u_n1 + R_num(3) * u_n2);

    % Compute S(z)/R(z) * phi
    S_R = (S_num(1) * phi + S_num(2) * y_n1 + S_num(3) *
       y_n2);
    upid=single((T_R - S_R) / R_num(1));
    u_n2 = single(u_n1);
    u_n1 = upid;
    % Control output
     u = single(uff+upid);
    if u>max_voltage
        u=max_voltage;
    elseif u<-max_voltage
         u=-max_voltage;

    end
    % Update persistent variables
    r_n2 = r_n1;
    r_n1 = p_ref;
    y_n2 = y_n1;
    y_n1 = phi;
```

```
end
```

2.2 Comparing Embedded Controller with Block Diagram Approach

Figure 10 illustrates the trajectory control responses of the embedded function block and the ordinary Simulink block, which exhibit similar performance.

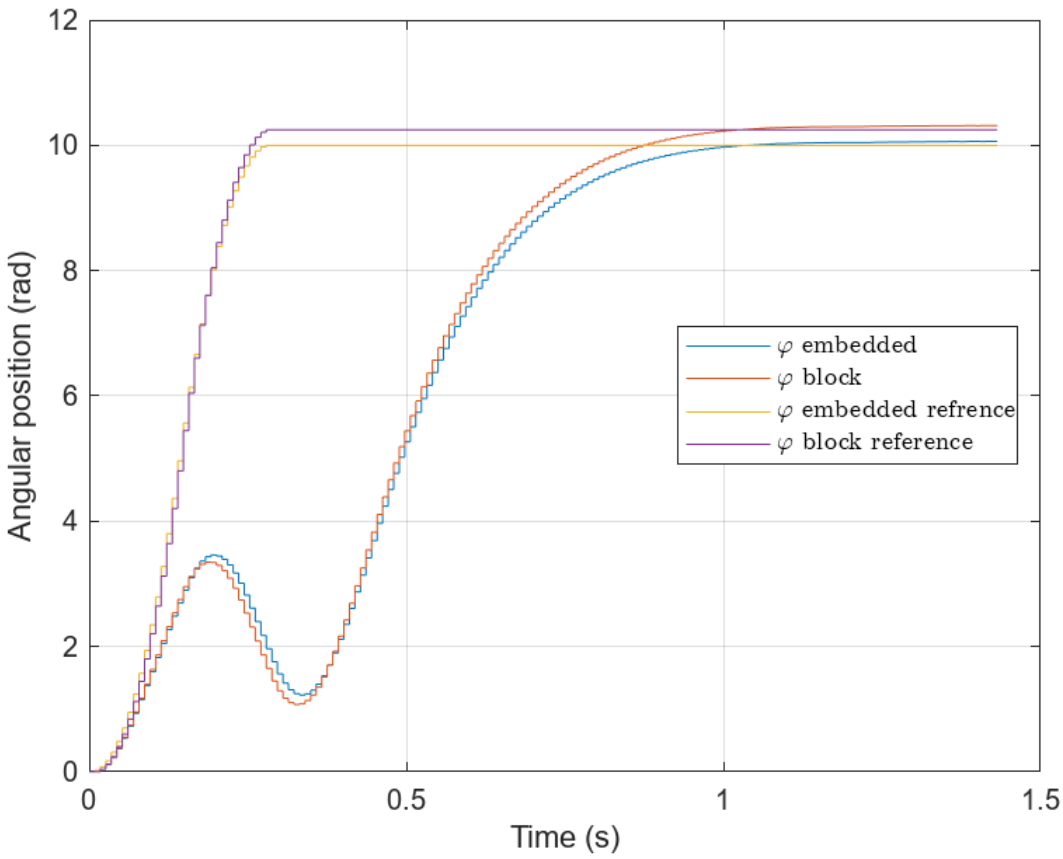


Figure 10: Trajectory path for 10 *rad* set point, comparing simulation results using embedded controller and block diagram controller.

The main difference is observed in where the trajectory position settles. As explained in the previous section, the trajectory using Simulink blocks overshoots. However, this is improved with the embedded controller. It is likely because the embedded controller does not require discrete time integration that can cause an accumulative error, instead, it calculates exactly the needed output for each time step. Despite these differences in trajectory handling, the control outputs from both approaches are nearly the same, as illustrated in Figure 11.

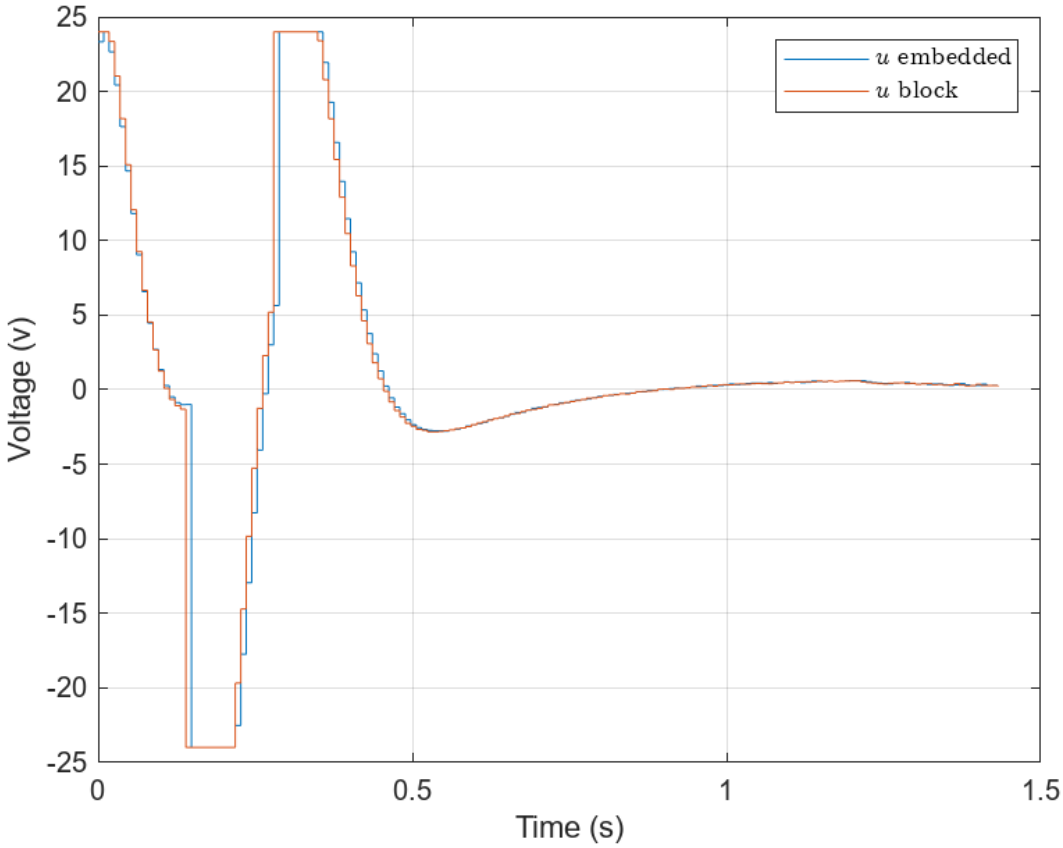


Figure 11: Control output for 10 *rad* set point, comparing simulation results using embedded controller and block diagram controller.

A reason for the discrepancy between the block simulation and the embedded simulation trajectories is the data type used. The embedded controller employs single precision while the block diagram controller uses double precision. This variation in precision causes the control values to be approximated differently by the two controllers.

As the set point value increases, the differences between the block diagram and the embedded controller approach become less noticeable, as shown in Figure 12.

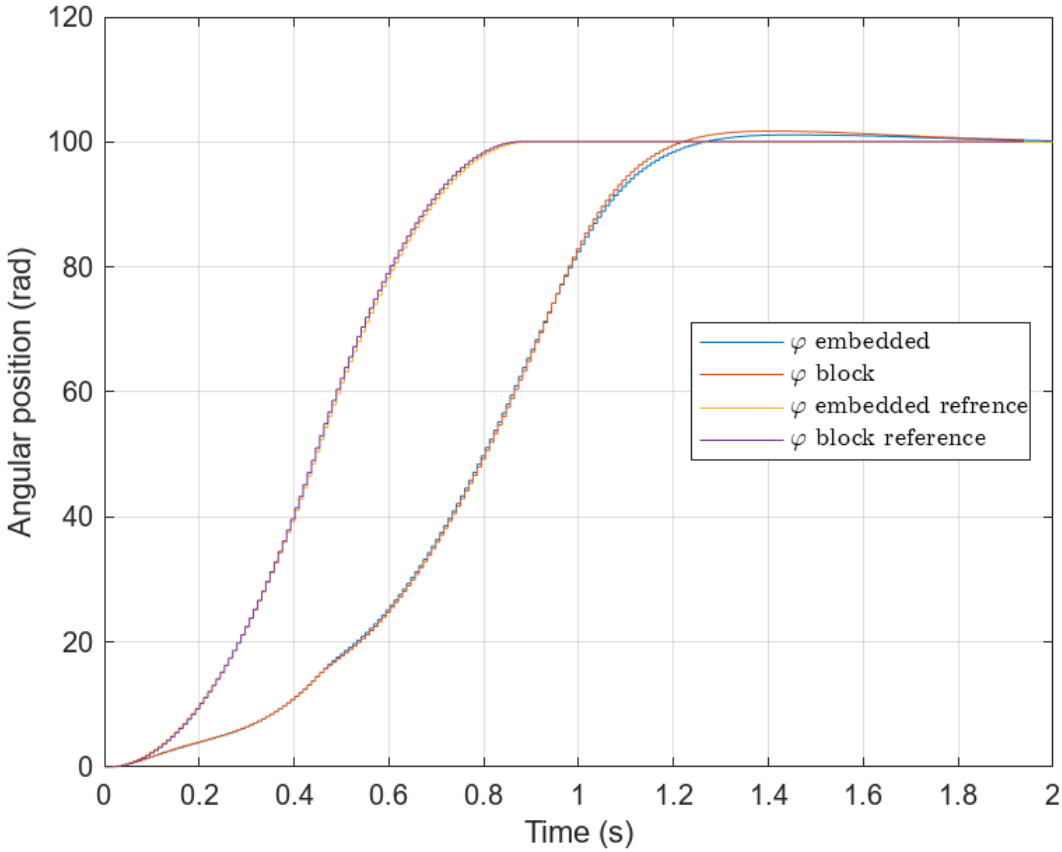


Figure 12: Trajectory path for 100 *rad* set point, comparing simulation results using embedded controller and block diagram controller.

However, the control output still differs as shown in Figure 13.

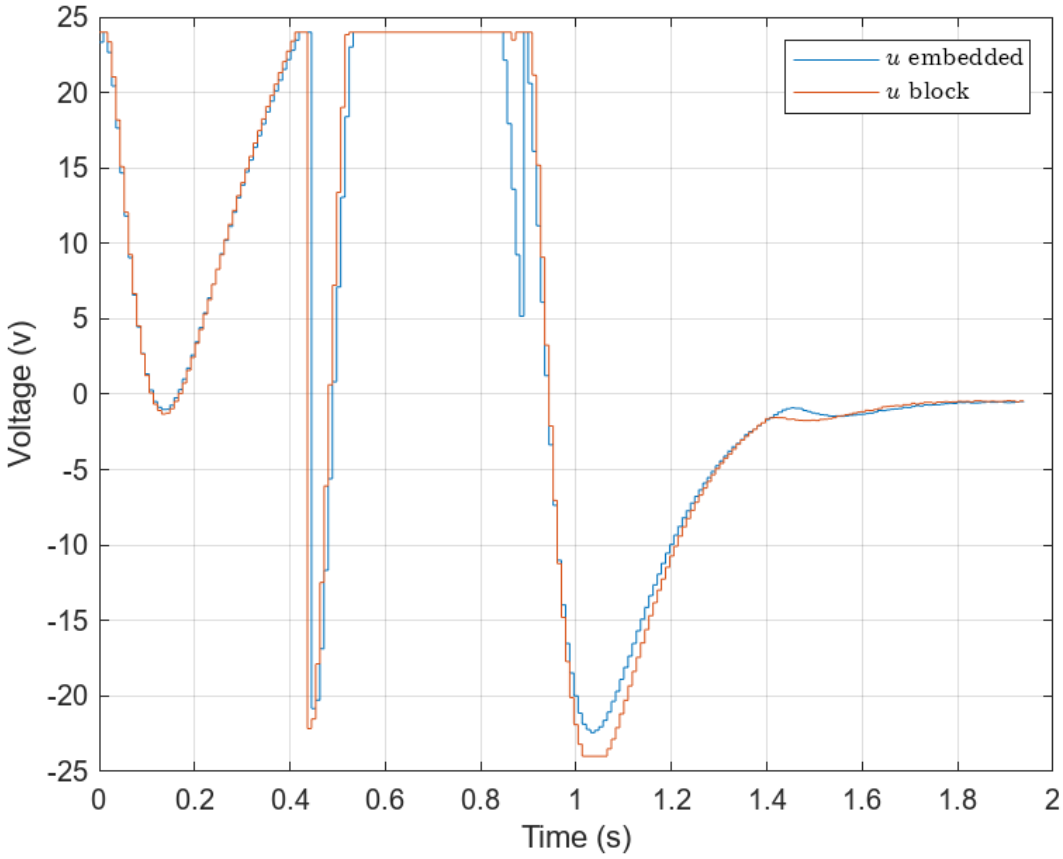


Figure 13: Control output for 100 *rad* set point, comparing simulation results using embedded controller and block diagram controller.

Noticeable differences emerge after 0.9 seconds. Although the reason for the discrepancies between the two curves can not be fully explained, they appear to occur when the trajectory paths reach their set point value and the PID takes over.

3 Robustness to Parameter Uncertainty and Sensor Noise

3.1 DC motor

The compare the Sensitivity function, $S_e(s)$, and Complimentary Sensitivity function, $T_e(s)$, between different values of A_m and A_o, they first needed to be determined. Substituting in values for A, B and R into the equations

$$S_e(s) = \frac{AR}{A_m A_o}, \tag{3.1}$$

$$T_e(s) = \frac{BS}{A_m A_o}, \tag{3.2}$$

they can be analyzed for different values of A_m and A_o. For the following Bode diagram, the system pole A_m was chosen to the be same as in previous tasks, following the specifications. The comparisons used A_o poles 2 times faster than A_m, as well as 4 times faster than A_m, as specified in the task description.

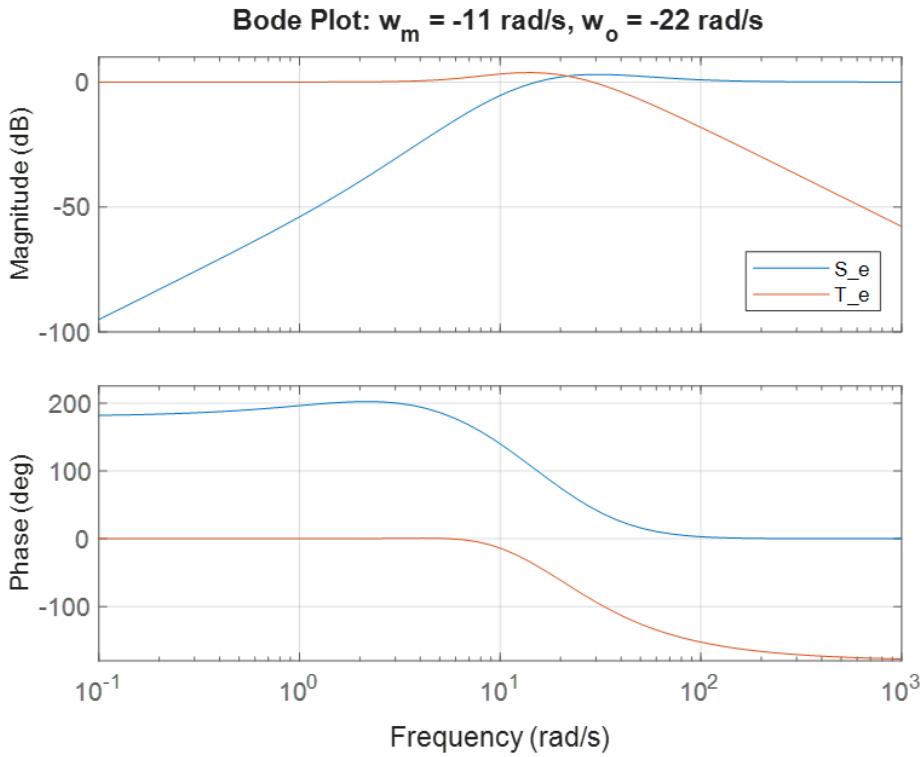


Figure 14: Bode diagram comparing $S_e(s)$ and $T_e(s)$ using ω_m = -11 rad/s and ω_o = -22 rad/s.

In Figure 14, the crossover frequency between $S_e(s)$ and $T_e(s)$ appears roughly at the observer pole. We also notice that $S_e(s)$ has high-pass characteristics, while $T_e(s)$ has low-pass characteristics. It shows the trade-offs in controller design. While choosing A_o poles at a low frequency ensures attenuated disturbances due to the high-pass characteristics of $S_e(s)$, it ultimately means the ability to reject sensor noise reduces, due to the low-pass characteristics of $T_e(s)$. The position of the observer poles affects how the system balances these two characteristics, which is further shown in Figure 15 below.

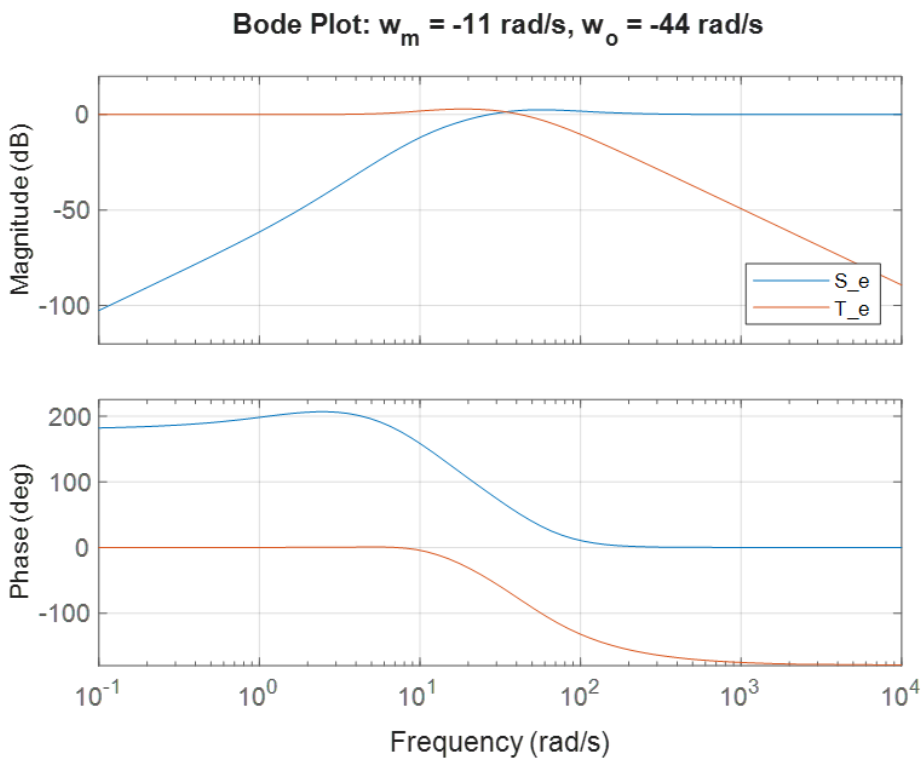


Figure 15: Bode diagram comparing $S_e(s)$ and $T_e(s)$ using ω_m = -11 rad/s and ω_o = -44 rad/s.

Two conclusions can be drawn when comparing the two plots and the system responses from previous tasks. First: A_o determines the response for disturbances and noise, and A_m and t_0 determines the response for the reference. Placing the observer poles at higher frequencies and raising the crossover point improves the system's speed, responsiveness and robustness to modeling errors. However, as can be seen, it also makes the system more sensitive to noise. Since the process parameters, such as d_m and J, are not perfectly known and may change over time, particularly friction, and given that noise levels can vary depending on the environment the system operates in, it calls for a careful balance in the placement of the A_o poles to achieve an optimal trade-off between noise sensitivity and model error rejection. Second: another important factor for choosing the observer pole

was the sample time, T_s. Different sample times lead to variations in the discrete-time implementation of the controller, which can affect stability and performance. A smaller T_s allows for higher observer pole placement and better tracking of fast system dynamics, but it can require more computational resources than the hardware allows for. Conversely, larger T_s limits the observer pole placement and may reduce responsiveness, potentially degrading overall control performance. This highlights the need to carefully balance observer pole placement with the chosen sample time to achieve optimal performance.

3.2 Hydraulic Actuator

This subsection presents the design of a velocity controller for a valve-controlled hydraulic cylinder using a nonlinear model and pole-placement method. It examines how different frequencies of $A_m(s)$ and $A_0(s)$ affect the control performance, particularly under varying conditions such as external force, sensor noise, and increased mass.

3.2.1 Controller Design

The transfer function of the linearized plant is given by:

$$G_p(s) = \frac{3.612 \times 10^7}{s^2 + 56.81s + 1.886 \times 10^5}. \tag{3.3}$$

Accordingly, a PID controller with a low-pass filter is chosen, resulting in four equations with four unknowns. Both A_m and A_0 are second order. The poles of A_0 and A_m are assumed to be real and identical, with $p_0 = p_{01} = p_{02}$ and $p_m = p_{m1} = p_{m2}$, respectively.

Initially, the same pole frequency is selected for both A_m and A_0 ($p_0 = p_m$). Starting from 100 rad/s, the frequency is gradually increased. Negative PID gains were observed at lower frequencies, which would destabilize the system by reversing control actions. At 430 rad/s, all PID gains become positive, and the system is stable. Further increasing the frequency beyond 500 rad/s results in small improvements. Therefore, A_m is determined as:

$$A_m = (s + 500)^2. \tag{3.4}$$

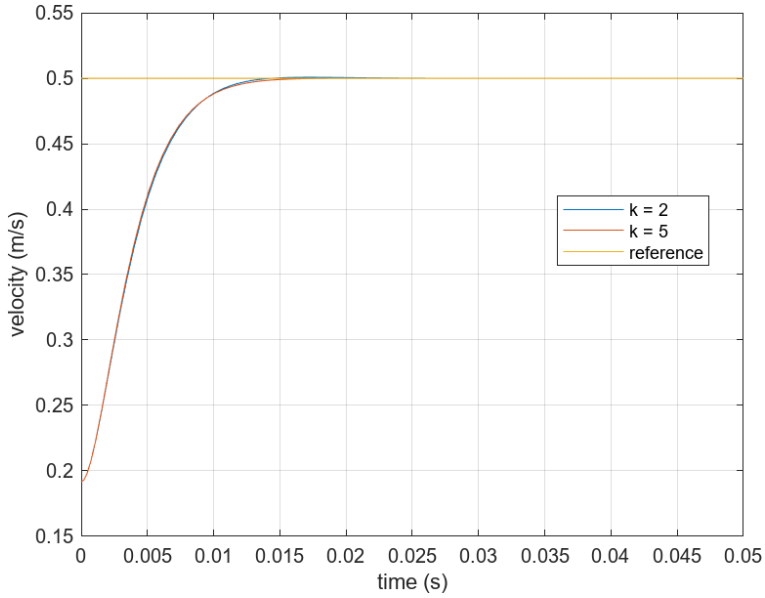
Next, the frequency of A_0 is selected as a factor (k) of the frequency in A_m:

$$s_0 = k \cdot s_p. \tag{3.5}$$

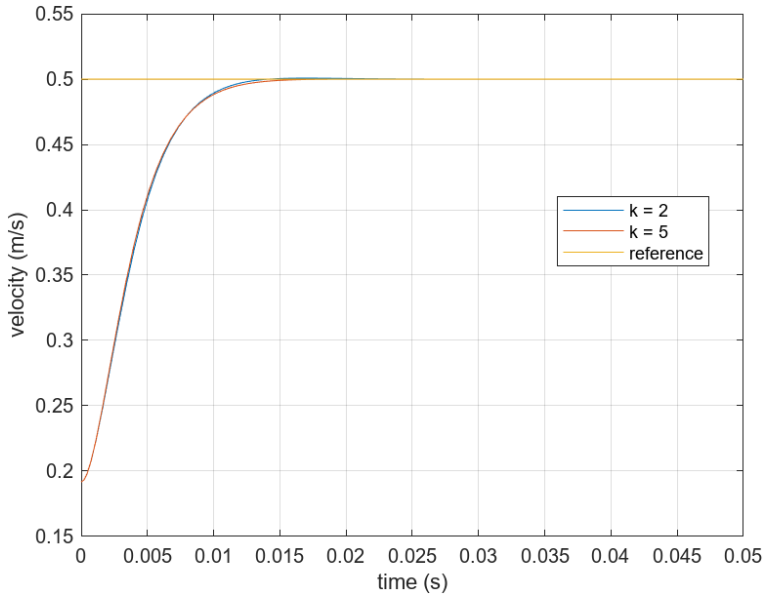
Step responses are evaluated, where the reference (r_v) is a step input of 0.5 m/s. Starting with $k = 1$ and gradually increasing it to 2, satisfactory performance is achieved, with a short rise time and subtle overshoot. For $k = 5$, the overshoot almost disappeared and the rise time is slightly shorter, indicating improved system performance. The resulting step responses are shown in Figure 18a) for mass $m = 100$ kg and Figure 18b) for $m = 200$ kg. Rise time and overshoot are summarized in Table 1.

Table 1: Comparison of rise time and overshoot

	m = 100 kg		**m = 200 kg**	
	Rise Time (ms)	**Overshoot (%)**	**Rise Time (ms)**	**Overshoot (%)**
k = 2	14.848	0.147	14.718	0.152
k = 5	14.704	0.002	14.614	0.002



(a) mass = 100 kg



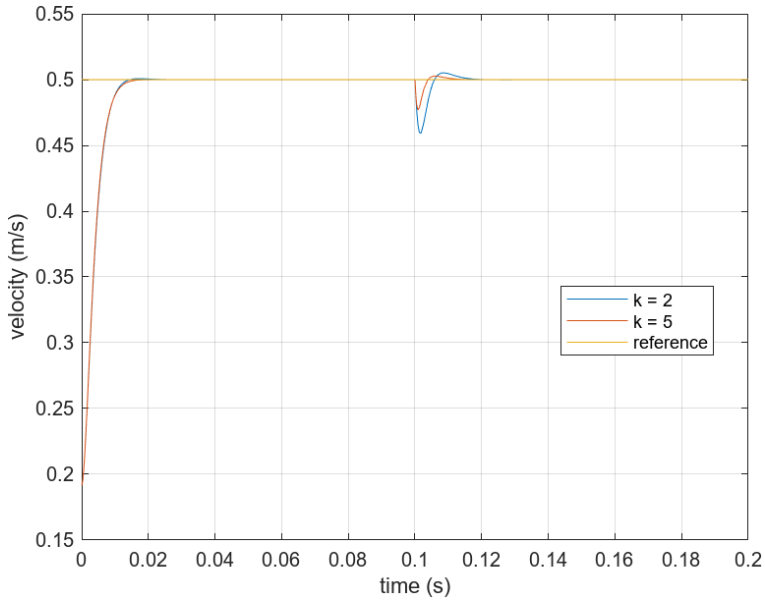
(b) mass = 200 kg

Figure 16: Comparison of step response under applied force.

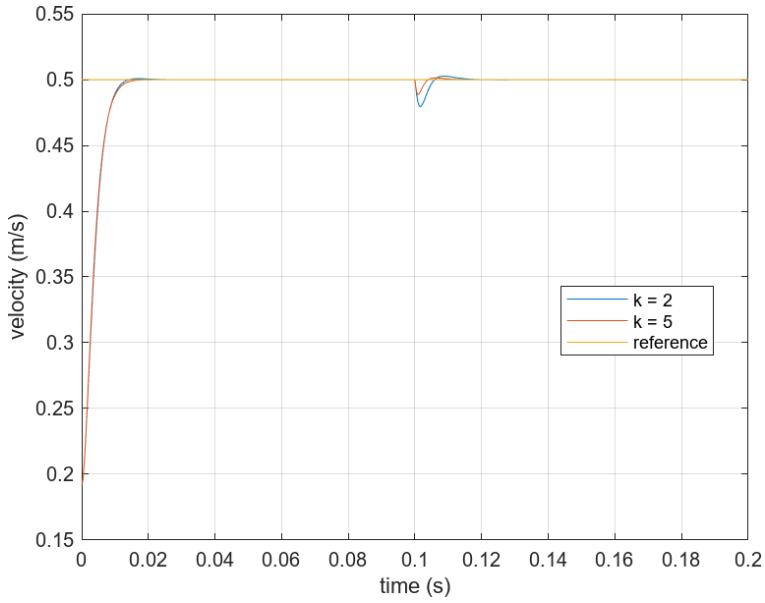
3.2.2 Response under External Force

A step disturbance force (f_e) of 5000 N is applied at 0.1 s after the velocity stabilizes. The resulting step responses are shown in Figure 17. The controller with faster observer $k = 5$ has better disturbance rejection, with less fluctuation and faster recovery.

When the mass is increased from 100 kg to 200 kg, the disturbance rejection improves for both k values due to the higher inertia of the system. The larger mass reduces the magnitude of velocity fluctuations caused by the disturbance, leading to a more robust response.



(a) Mass = 100 kg



(b) Mass = 200 kg

Figure 17: Comparison of step response under applied force $f_e = 5000\ N$ at 0.1 s.

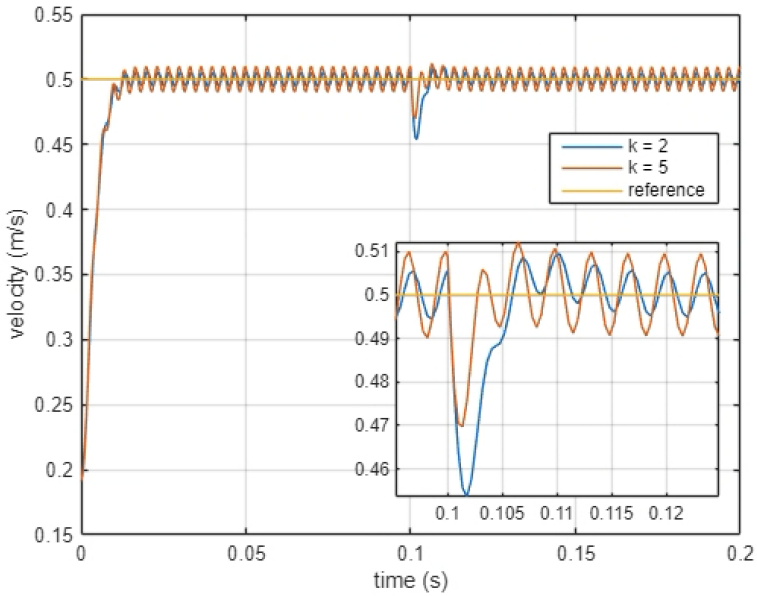
3.2.3 Response under Sensor Noise

A sinusoidal sensor noise with an amplitude of 0.01 m/s is added to the output to evaluate the system's performance under noise for different scaling factors and masses. The noise frequency f_n is set to 300 Hz. The resulting step responses are shown in Figure 18. Rise time and overshoot under noise are summarized in Table 2.

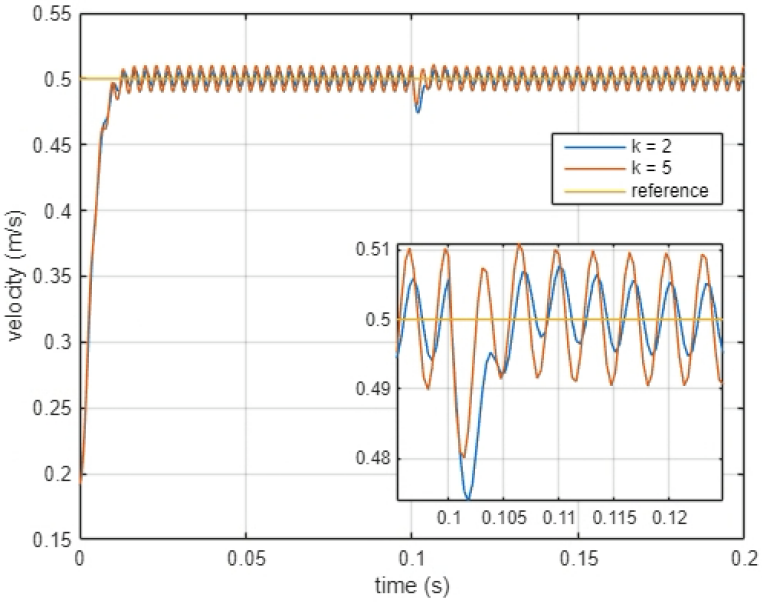
For $k = 5$, the system shows shorter rise times and smaller overshoot compared to $k = 2$. The controller with $k = 2$ performs better at rejecting high-frequency noise from the step response. When the mass increases twice, which could simulate the situation of the machine picking up a payload, the faster observer $k = 5$ performs better at rejecting modeling errors.

Table 2: Comparison of rise time and overshoot under disturbance

	m = 100 kg		**m = 200 kg**	
	Rise Time (ms)	**Overshoot (%)**	**Rise Time (ms)**	**Overshoot (%)**
k = 2	18.195	0.889	22.786	0.496
k = 5	17.844	0.707	22.503	0.400



(a) mass = 100 kg



(b) mass = 200 kg

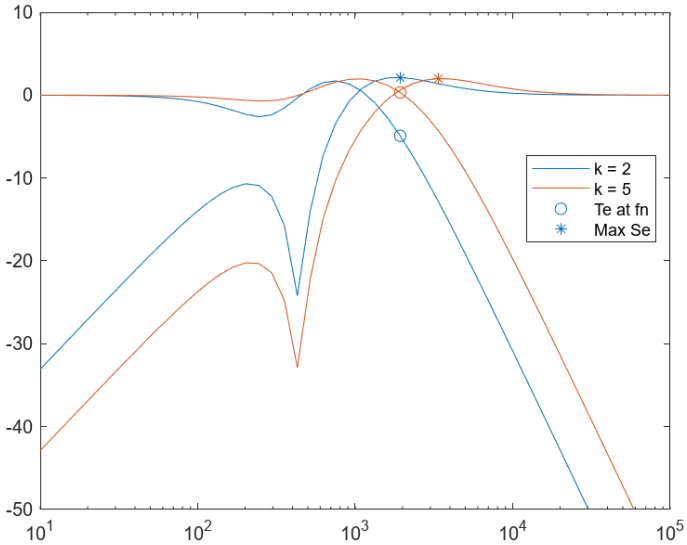
Figure 18: Comparison of step response with sensor noise.

The sensitivity function S_e and the complementary sensitivity function T_e are given by:

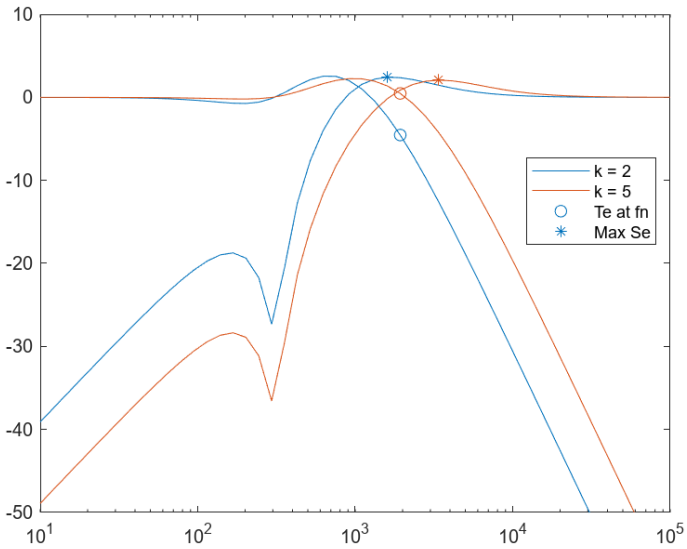
$$S_e(s) = \frac{AR}{AR + BS} \tag{3.6}$$

$$T_e(s) = 1 - S_e(s) = \frac{BS}{AR + BS}. \tag{3.7}$$

The bode plots for S_e and T_e are shown in Figure 19. The maximal S_e values and the T_e values at the noise frequency $f_n = 300\ Hz$ are summarized in Table 3, which further validates the results. Faster observer $k = 5$ with smaller mass has bigger T_e at f_n, indicating worse ability in rejecting sensor noise. Slower observer $k = 2$ with larger mass has bigger maximal S_e, showing higher sensitivity and worse ability in handling modeling errors.



(a) mass = 100 kg



(b) mass = 200 kg

Figure 19: Comparison of Magnitude Bode Diagram under sensor noise.

Table 3: Comparison of maximal Se and Te at noise frequency fn

	m = 100 kg		**m = 200 kg**	
	Max Se	**Te at fn**	**Max Se**	**Te at fn**
k = 2	2.14	-4.90	2.43	-4.54
k = 5	1.99	-0.33	2.07	-0.47

3.2.4 Discussion

The simulation results show that the faster observer achieves a shorter rise time, smaller overshoot, and better disturbance rejection. However, it exhibits greater sensitivity to noise and requires a much shorter sampling time.

When subjected to a step disturbance f_e, $k = 2$ provides quicker recovery and less fluctuation. Increasing the mass to 200 kg enhances the system's robustness due to higher inertia, reducing fluctuations for both k values.

Under sensor noise, $k = 2$ demonstrates better noise attenuation, as indicated by lower T_e values at the noise frequency and reduced oscillations in the step response. On the other hand, $k = 5$ offers better modeling error rejection, reflected in a lower maximal S_e. These results highlight the need for a compromise in regulator design to balance performance trade-offs.